Basic (your first graph) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `first_graph_practice.py`: this is the smallest possible LangGraph graph — a one-field `TypedDict` state, two nodes (each a plain function that takes the state and returns updates to it), and one `add_edge` connecting them — deliberately tiny so the 3 real ingredients (state, nodes, edges) are visible on their own, before Doc07's whole agent loop gets rebuilt as a graph in the next exercise. **If not:** `agent_loop_to_graph` would be the first place you ever saw `StateGraph` / `add_node` / `add_edge` at all, with a much bigger graph to learn them from.

Solution

```
# first_graph_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    message: str

def node_one(state: GraphState) -> dict:
    return {"message": "Hello"}

def node_two(state: GraphState) -> dict:
    return {"message": state["message"] + ", World!"}

builder = StateGraph(GraphState)
builder.add_node("node_one", node_one)
builder.add_node("node_two", node_two)
builder.add_edge(START, "node_one")
builder.add_edge("node_one", "node_two")
builder.add_edge("node_two", END)

graph = builder.compile()
result = graph.invoke({"message": "start"})
print(result)
```

Expected output:

```
{'message': 'Hello, World!'}
```